

```
$ spring run TransactionLab --propagation=REQUIRED
```

# @Transactional 事務管理

// 要嘛全做，要嘛全不做 — 而且命名決定你的命運

**begin**

開啟事務

**commit**

觸發 flush

**rollback**

當作沒發生

**proxy**

幕後功臣

\$ cat agenda.md

# 今日路線

## 01 / 是什麼

銀行轉帳與三大職責

## 02 / 命名即命運

全域攔截器的兩套合約

## 03 / AOP 代理

替身演員如何開關事務

## 04 / 陷阱與實踐

自我呼叫、rollback 規則、FAQ

目標：不再問「為什麼我的 @Transactional 沒生效？」

PART 01

# @Transactional 是什麼？

先從一筆會失敗的轉帳說起

# 銀行轉帳的四個步驟

## 1. 開門

開啟事務

## 2. A 扣錢

UPDATE accounts...

## 3. B 加錢

UPDATE accounts...

## 4. 完成

Commit

WHAT IF STEP 3 FAILS?

錢不能憑空消失 —— 銀行說「當作沒發生過」 (**Rollback**)，把錢還給 A。

@Transactional 就是那個確保「**要嘛全做，要嘛全不做**」的銀行經理。

# 三大職責（外加一個隱藏版）

| 職責   | 說明                                  | 比喻                |
|------|-------------------------------------|-------------------|
| 開啟合約 | 方法開始時確保有 <code>EntityManager</code> | 婚禮開場白             |
| 定義行為 | 「這是不可分割的業務」                         | 「你願意嗎？」           |
| 觸發回滾 | 出錯時通知資料庫回滾                          | 「我反對！」 → 婚禮取消     |
| 強制同步 | 正常結束時觸發 <code>flush()</code>        | 「我宣布你們...」 → 蓋章生效 |

最重要的一點：方法正常結束 → `commit` → 強制 `flush()`。這就是為什麼你改了 `Entity` 沒呼叫 `save()`，資料還是存進去了。

PART 02

# 命名即命運

全域事務攔截器的兩套合約

# 寫入合約

```
1 // 這些方法名開頭
2 // 自動獲得事務保護
3 add*, save*, create*,
4 update*, delete*
```

PROPAGATION\_REQUIRED

「必須有事務！沒有就開一個，有就加入」

遇 `RuntimeException` → 回滾

# 唯讀合約

```
1 // 這些方法名開頭
2 // 只能讀不能寫
3 find*, get*, search*,
4 getCount*, * (其他所有)
```

PROPAGATION\_NOT\_SUPPORTED

「不需要事務，我只是來看看」

`setReadOnly(true)` : 只看不摸

# 命名陷阱

```
1 // ✅ 有事務保護
2 public void saveUser() { ... }
3 public void updateOrder() { ... }
4
5 // ❌ 沒有事務保護 (即使你加了 @Transactional)
6 public void processUser() { ... } // 不符合命名規則
7 public void handleOrder() { ... } // 被 * 攔截成唯讀
```

## PRIORITY

就算手動打上 `@Transactional`，攔截器的優先級更高，`process*` 還是被當成唯讀處理。

方法命名決定你有沒有事務保護

CHECKPOINT\_01

關於全域事務攔截器，哪一句正確？

A processUser() 加上 @Transactional，就一定有寫入事務

B 同類別內 this.saveUser() 呼叫，也會經過 AOP 代理

C 攔截器以方法名決定事務行為，process\* 可能被當成唯讀

PART 03

# AOP 代理：幕後功臣

Spring 沒有改你的程式碼，它派了替身

# 替身演員的工作流程



PART 04

# 陷阱與最佳實踐

90% 的「沒生效」都在這裡

# 自我呼叫陷阱

```
1  @Service
2  public class UserService {
3
4      // 🚨 沒有獨立事務！
5      public void processAndSave(User user) {
6          validateUser(user);
7          saveUser(user); // this.saveUser()
8      }
9
10     public void saveUser(User user) {
11         userRepository.save(user);
12     }
13 }
```

## WHY

`this.saveUser()` 呼叫的是自己，不是代理物件 —— AOP 完全沒機會插手。

解法 1：把方法拆到不同 Service

解法 2：注入自己，用 `self.saveUser()`

# 「為什麼沒生效」四大原因

## 01 命名不符

攔截器規則優先，process\* 變唯讀

## 02 自我呼叫

this. 繞過代理

## 03 不是 public

private 是內心戲，代理看不到

## 04 Checked Exception

預設只回滾 RuntimeException

\$ 下次事務沒生效，照這四條依序排查，命中率 90%。

# 最佳實踐

## DO

- ✓ 遵守命名規則：save\* / update\* / delete\*
- ✓ 事務方法保持簡短，鎖越短越好
- ✓ 只在 Service 層使用

## DON'T

- × 事務方法太長，其他請求排隊
- × 在迴圈裡開事務，效能爆炸
- × 混用不同資料源（那是分散式事務的故事）

# 今日重點

## 生命週期管理者

開啟、監控、commit (觸發 flush) 或 rollback

## 命名決定命運

攔截器依方法名分配寫入／唯讀合約

## 代理是幕後功臣

this. 呼叫與 private 方法都繞過它

## 唯讀優化

查詢用 find\* / get\*，省鎖省 dirty checking

要嘛全做，要嘛全不做 — 而且方法名說了算。

# Transaction committed.

下一步：JPA 持久化上下文，把 flush 與 dirty checking 釐清

```
$ spring run TransactionLab --status  
begin → invoke → commit(flush) | rollback
```